85. Динамическое выделение памяти

Язык C++ поддерживает три основных типа **выделения** (или **«распределения»**) **памяти**, с двумя из которых, мы уже знакомы:

- **Статическое выделение памяти** выполняется для **статических** и **глобальных** переменных. Память выделяется один раз (при запуске программы) и сохраняется на протяжении работы всей программы.
- **Автоматическое выделение памяти** выполняется для **параметров функции** и **локальных переменных**. Память выделяется при входе в блок, в котором находятся эти переменные, и удаляется при выходе из него.
- **Динамическое выделение памяти** является темой этого урока.

Оглавление:

1. Динамическое выделение переменных
2. Как работает динамическое выделение памяти?
3. Освобождение памяти
4. Висячие указатели
5. Оператор new
6. Нулевые указатели и динамическое выделение памяти
7. Утечка памяти
8. Заключение

Динамическое выделение переменных

Как статическое, так и автоматическое распределение памяти имеют два общих свойства:

- Размер переменной/массива должен быть известен во время компиляции.
- Выделение и освобождение памяти происходит автоматически (когда переменная создается/уничтожается).

В большинстве случаев с этим всё ОК. Однако, когда дело доходит до работы с пользовательским вводом, то эти ограничения могут привести к проблемам.

Например, при использовании строки для хранения имени пользователя, мы не знаем наперед насколько длинным оно будет, пока пользователь его не введет. Или нам нужно создать игру с непостоянным количеством монстров (во время игры один монстр умирает, другие появляются, пытаясь, таким образом, убить игрока).

Если нам нужно объявить размер всех переменных во время компиляции, то самое лучшее, что мы можем сделать — это попытаться угадать их максимальный размер, надеясь, что этого будет достаточно:

```
char name[30]; // assume name is no longer than 30 characters
Monster monster[10]; // assume there are no more than 10 monsters
Polygon rendering[5000]; // assume we don't need more than 5000
```

Это плохое решение, по крайней мере, по трем причинам:

Во-первых, тратится память, если переменная фактически не используется или используется, но не все. Например, если мы выделим 30 символов для каждого имени, но имена в среднем будут занимать по 15 символов, то потребление памяти получится в два раза больше, чем нам нужно на самом деле. Или рассмотрим массив `rendering`: если он используют только 20 000 полигонов, то память для других 20 000 полигонов фактически растратится впустую (т.е. не используется)

Во-вторых, память для большинства обычных переменных (включая фиксированные массивы) выделяется из специального резервуара памяти — **стека**. Объем памяти стека в программе, как правило, меньше: в Visual Studio он по умолчанию равен 1МБ. Если вы превысите это значение, то произойдет *переполнение стека*, и операционная система автоматически завершает выполнение вашей программы.

В Visual Studio это можно проверить, запустив следующий фрагмент кода:

```
int main()
{
    int array[1000000000]; // assume 1 million will overflow stack
}
```

Лимит в 1МБ памяти может быть проблематичным для многих программ, особенно где используется графика.

В-третьих, и самое главное, это может привести к искусственным ограничениям и/или переполнению массива. Что произойдет, если пользователь попытается прочесть 500 записей с диска, но мы выделили память максимум для 400? Либо мы выведем пользователю ошибку, что максимальное количество записей — 400, либо (в худшем случае) выполнится переполнение массива и затем что-то очень некрасивое.

К счастью, эти проблемы легко устраняется с помощью динамического выделения памяти. **Динамическое выделение памяти** — это запрос памяти из операционной системы запущенным программами по мере необходимости. Эта память не выделяется из ограниченной памяти стека программы, а выделяется из гораздо большего хранилища, управляемого операционной системой — **кучи**. На современных компьютерах куча может составлять гигабайты памяти.

Для динамического выделения памяти одной переменной используется **оператор new**:

```
new int; // request an integer from the operating system
```

В примере, приведенном выше, мы запрашиваем выделение памяти для целочисленной переменной из операционной системы. Оператор new возвращает **указатель**, содержащий адрес выделенной памяти.

Для доступа к выделенной памяти создается указатель:

```
int *ptr = new int; // dynamically allocate an integer
```

Затем мы можем разыменовать указатель для получения значения:

```
*ptr = 7; // assign value to dynamically allocated integer
```

Вот один из случаев, когда указатели полезны. Без указателя с адресом на только что выделенную память у нас не было бы способа получить доступ к ней.

Как работает динамическое выделение памяти?

На вашем компьютере имеется память (возможно, большая её часть), которая доступна для использования программами. При запуске программы ваша операционная система загружает эту программу в некоторую часть этой памяти. И эта память, используемая вашей программой, разделена на несколько частей, каждая из которых выполняет определенную задачу. Одна часть содержит ваш код, другая используется для выполнения обычных операций (отслеживание вызываемых функций, создание и уничтожение глобальных и локальных переменных и т.д.). Мы поговорим об этом чуть позже. Тем не менее, большая часть доступной памяти компьютера просто находится в ожидании запросов на выделение от программ.

Когда вы динамически выделяете память, то вы просите операционную систему зарезервировать часть этой памяти для использования вашей программой. Если ОС может выполнить этот запрос, то возвращается адрес этой памяти обратно в вашу программу. С этого момента и в дальнейшем ваша программа может использовать эту память, как только пожелает. Когда вы уже выполнили с этой памятью всё, что было необходимо, то ей нужно вернуть обратно в операционную систему, для распределения между другими запросами.

В отличие от статических или автоматического выделения памяти, программа самостоятельно отвечает за запрос и обратный возврат динамически выделенной памяти.

Освобождение памяти

Когда вы динамически выделяете переменную, то вы также можете её инициализировать посредством прямой инициализации или uniform-инициализации (в C++11):

```
int *ptr1 = new int (5); // initialize dynamic variable
int *ptr2 = new int { 6 }; // initialize dynamic variable
```

Когда уже всё, что требовалось, выполнено с динамически выделенной переменной — нужно явно указать для C++ освободить эту память. Для переменных это выполняется с помощью **оператора delete**:

```
int *ptr = new int;
*ptr = 7;
// stuff done with ptr here
delete ptr; // return memory to the operating system. ptr is now a dangling pointer
ptr = 0; // set ptr to a null pointer (we'don't want it to point to deleted memory anymore)
```

Оператор delete на самом деле ничего не удаляет. Он просто возвращает память, которая была выделена ранее, обратно в операционную систему. Затем операционная система может перераспределить эту память другому приложению (или этому же снова).

Хотя может показаться, что мы удаляем переменную, но это не так! Переменная-указатель по-прежнему имеет ту же область видимости, что и раньше, и ей можно присвоить новое значение, как и любой другой переменной.

Обратите внимание, удаление указателя, не указывающего на динамически выделенную память, может привести к проблемам.

Висячие указатели

Язык C++ не предоставляет никаких гарантий относительно того, что произойдет с содержимым освобожденной памяти или со значением указателя. В большинстве случаев, память, возвращаемая операционной системе, будет содержать те же значения, которые были у нее до освобождения, а указатель так и останется указывать на только что освобожденную (удаленную) память.

Указатель, указывающий на освобожденную память, называется **висячим указателем**. Разыменование или удаление висячего указателя приведет к непредсказуемым результатам. Рассмотрим следующую программу:

```
#include <iostream>

int main()
{
    int *ptr = new int; // dynamically allocate an integer variable
    *ptr = 5; // assign value to dynamically allocated integer

    delete ptr; // return memory to the operating system. ptr is now a dangling pointer

    std::cout << *ptr; // dereferencing dangling pointer will produce a memory access error
    delete ptr; // calling delete on a dangling pointer may lead to memory corruption

    return 0;
}
```

В программе, приведенной выше, значение `8`, которое ранее было присвоено динамической переменной, после освобождения может и далее находиться там, и может и нет. Также, если что-то, что освобождается память уже могло быть выделена другому приложению (или для собственного использования операционной системы), и попытка доступа к ней приведет к тому, что операционная система автоматически прервет выполнение вашей программы.

Процесс освобождения памяти может также привести к и созданию нескольких висячих указателей. Рассмотрим следующий пример:

```
#include <iostream>

int main()
{
    int *ptr = new int; // dynamically allocate an integer variable
    int *otherPtr = ptr; // otherPtr points to the same memory as ptr

    delete ptr; // deallocate memory pointed to by ptr
    ptr = 0; // ptr points to null

    // ptr is now a null pointer

    // otherPtr is dangling

    return 0;
}
```

Есть несколько рекомендаций, которые могут здесь помочь:

- Во-первых, старайтесь избежать ситуаций, когда несколько указателей указывают на одну и ту же часть выделенной памяти. Если это невозможно, то выяснете, какой указатель из всех «владеет» памятью (и отвечает за её удаление), а какие указатели просто получают доступ к ней.
- Во-вторых, когда вы удаляете указатель, и, если он не выходит из области видимости сразу же после удаления, то его нужно сделать нулевым, т.е. присвоить значение `0` (или `nullptr` в C++11). Под «выходом из области видимости сразу же после удаления» имеется в виду, что вы удаляете указатель в самом конце блока, в котором он объявлен.

Правило: Присваивайте удаленным указателям значение 0 (или nullptr в C++11), если они не выходят из области видимости сразу же после удаления.

Оператор new

При запросе памяти из операционной системы в редких случаях она может быть не выделена (т.е. её может и не быть в наличии).

По умолчанию, если оператор new не сработал, память не выделилась, то генерируется исключение `bad_alloc`. Если это исключение будет неправильно обработано (а именно так и будет, поскольку мы еще не рассматривали исключения и их обработку), то программа просто прекратит свое выполнение (произойдет сбой) с ошибкой необработанного исключения.

Во многих случаях процесс генерации исключения оператором new (как и сбой программы) нежелателен, поэтому есть альтернативная форма оператора new, которая возвращает нулевой указатель, если память не может быть выделена. Нужно просто добавить **константу std::nothrow** между ключевым словом new и типом данных:

```
int *value = new (std::nothrow) int; // ask for an integer, and if we don't get it, set pointer to null
```

В примере, приведенном выше, если оператор new не возвратит указатель с динамически выделенной памятью, то возвратится нулевой указатель.

Разыменовывать его также не рекомендуется, так как это приведет к непредсказуемым результатам (скорее всего, к сбою в программе). Поэтому наилучшей практикой является проверка всех запросов на выделение памяти для обеспечения того, что эти запросы будут выполнены успешно и память выделится:

```
int *value = new (std::nothrow) int; // ask for an integer, and if we don't get it, set pointer to null
if (!value) // assume memory allocation failed
{
    // handle exceptional case here
    std::cout << "Could not allocate memory";
}
```

Поскольку не выделение памяти оператором new происходит крайне редко, то обычно программисты забывают выполнять эту проверку!

Нулевые указатели и динамическое выделение памяти

Нулевые указатели (указатели со значением `0` или `nullptr`) особенно полезны в процессе динамического выделения памяти. Их наличие как бы сообщает нам: «Этому указателю не выделено никакой памяти». А это, в свою очередь, можно использовать для выполнения условного выделения памяти:

```
// If we've not allocated memory to ptr yet, let's do so now.
if (!ptr)
    ptr = new int;
```

Удаление нулевого указателя ни на что не влияет. Таким образом, в следующем нет необходимости:

```
if (ptr)
    delete ptr;
```

Вместо этого вы можете просто написать:

```
delete ptr;
```

Если `ptr` не является нулевым, то динамически выделенная переменная будет удалена. Если значением указателя является нуль, то ничего не произойдет.

Утечка памяти

Динамически выделенная память не имеет области видимости, т.е. она остается выделенной до тех пор, пока не будет явно освобождена или пока ваша программа не завершит свое выполнение (и операционная система очистит все буфера памяти самостоятельно). Однако указатели, используемые для хранения динамически выделенных адресов памяти, следуют правилам области видимости обычных переменных. Это несоответствие может вызвать интересное поведение, например:

```
void doSomeThing()
{
    int *ptr = new int;
}
```

Здесь мы динамически выделяем целочисленную переменную, но никогда не освобождаем память через использование оператора delete. Поскольку указатели следуют всем тем же правилам, что и обычные переменные, то, когда функция завершит свое выполнение, `ptr` выйдет из области видимости. Поскольку `ptr` — это единственная переменная, хранящая адрес динамически выделенной целочисленной переменной, то, когда `ptr` уничтожится, большее не останется указателей на динамически выделенную память. Это означает, что программа «потеряет» адрес динамически выделенной памяти. И в результате эту динамически выделенную целочисленную переменную нельзя будет удалить.

Это называется **утечкой памяти**. Утечка памяти происходит, когда ваша программа теряет адрес некоторой динамически выделенной части памяти (например, переменной или массива), прежде чем вернуть её обратно в операционную систему. Когда это происходит, то программа уже не может удалить эту динамически выделенную память, поскольку больше не знает, где выделенная память находится. Операционная система также не может использовать эту память, поскольку считается, что она по-прежнему используется вашей программой.

Утечки памяти «съедают» свободную память во время выполнения программы, уменьшая количество доступной памяти не только для этой программы, но и для других программ также. Программы с серьезными проблемами с утечкой памяти могут «съесть» всю доступную память, в результате чего ваш компьютер будет медленнее работать или даже произойдет сбой. Только после того, как выполнение вашей программы завершится, операционная система сможет очистить и вернуть всю память, которая «утекла».

Хотя утечка памяти может возникнуть и из-за того, что указатель выходит за области видимости, возможны и другие способы, которые могут привести к утечке памяти. Например, если указателю, хранящему адрес динамически выделенной памяти, присвоить другое значение:

```
int value = 7;
int *ptr = new int; // assume memory allocation succeeds
ptr = &value;
```

Это легко решается удалением указателя перед операцией переприсваивания:

```
int value = 7;
int *ptr = new int; // assume memory allocation succeeds
delete ptr; // deallocate memory pointed to by ptr before it's overwritten
ptr = &value;
```

Кроме того, утечка памяти также может произойти и через двойное выделение памяти:

```
int *ptr = new int;
ptr = new int; // assume memory allocation succeeds
```

Адрес, возвращаемый из второго выделения памяти, перезатирает адрес из первого выделения. Следовательно, первое динамическое выделение становится утечкой памяти!

Точно так же этого можно избежать удалением указателя перед операцией переприсваивания.

Заключение

С помощью операторов new и delete можно динамически выделять отдельные переменные в программе. Динамически выделенная память не имеет области видимости и остается выделенной до тех пор, пока не произойдет её освобождение или пока программа не завершит свое выполнение. Будьте осторожны, не разыменовывайте висячие или нулевые указатели.

На следующем уроке мы рассмотрим использование операторов new и delete для выделения и удаления динамически выделенных массивов.